



Sichuan University -
Pittsburgh Institute

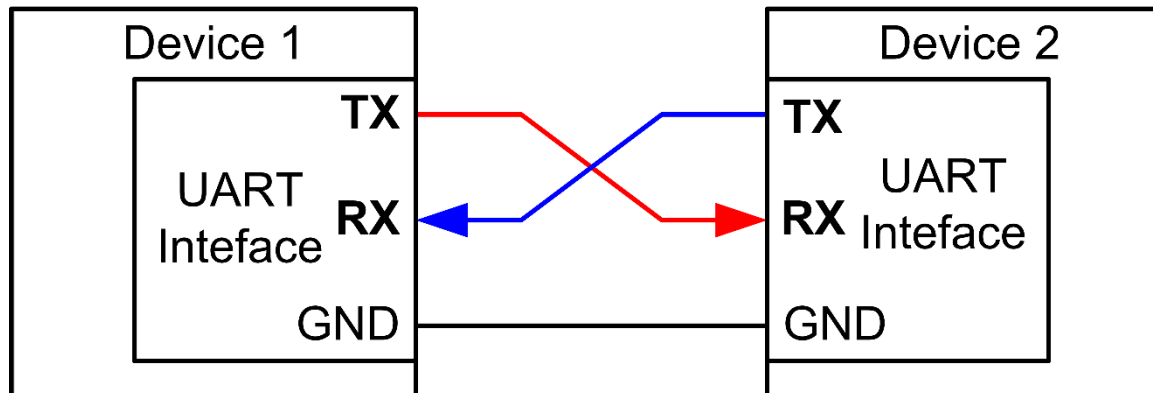
ECE0202 Embedded Processors & Interfacing + Lab

Sichuan University-Pittsburgh Institute



Universal Asynchronous Receiver and Transmitter (UART)

- Universal
 - UART can communicate with many different standards.
- Asynchronous
 - Sender provides no clock signal to receivers

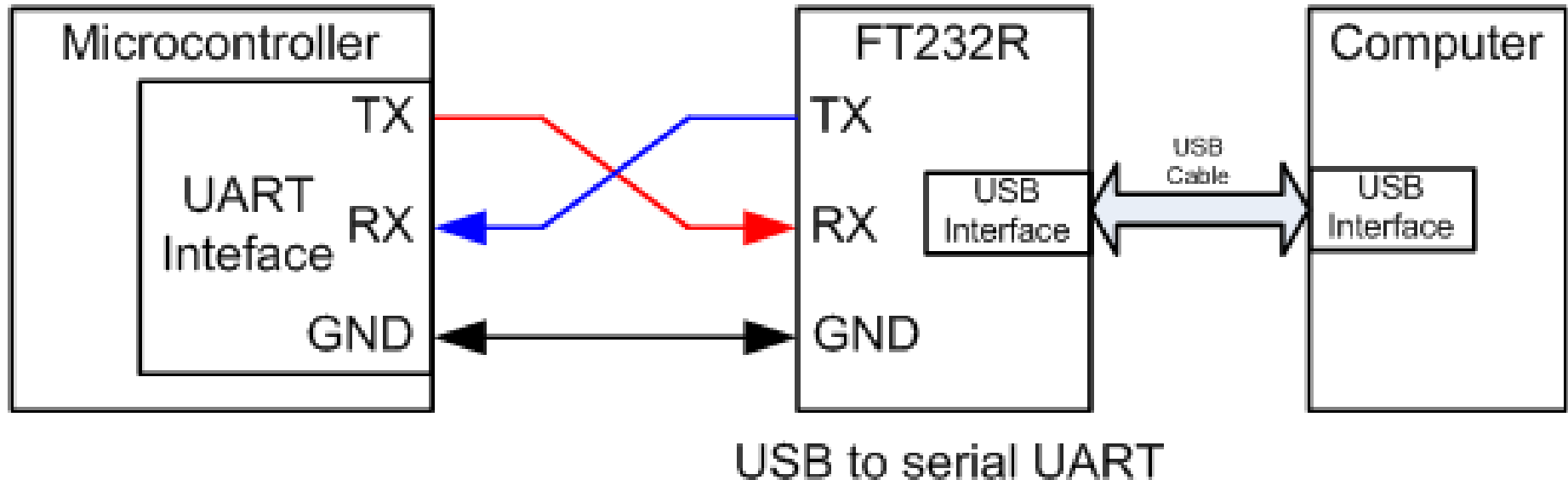


RS232

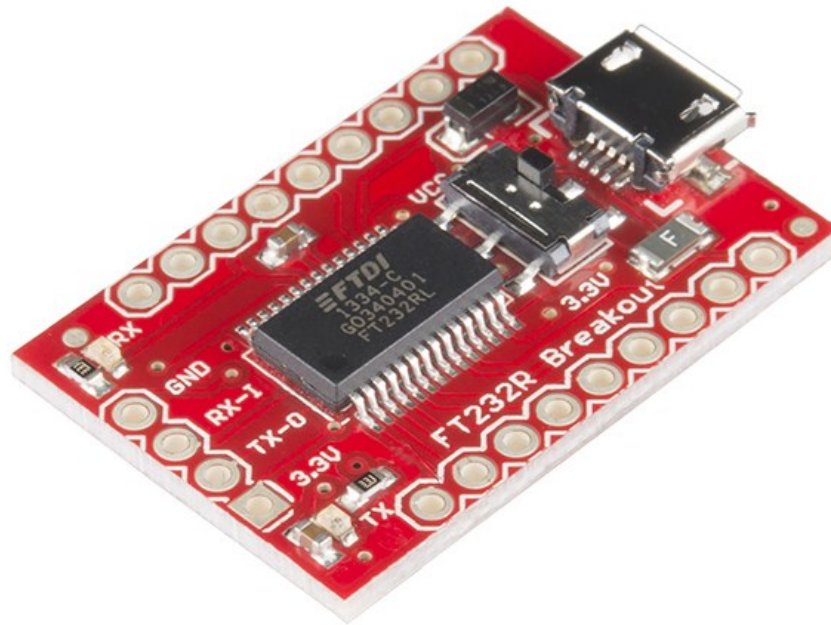


Connecting to PC

- FT232R converts the UART port to a standard USB interface



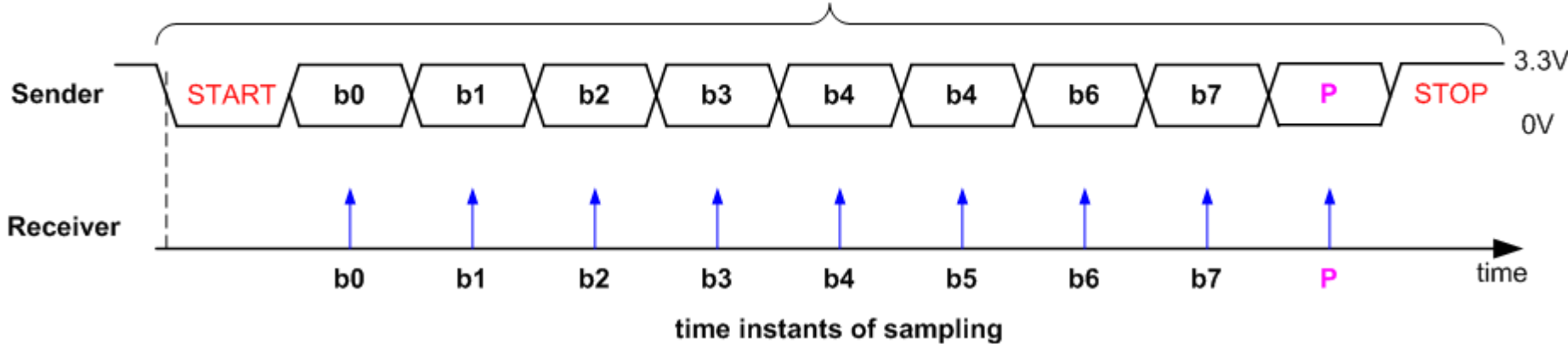
FT232R



<https://www.sparkfun.com/products/12731>

Data Frame

One Frame (1 start bit, 8 data bits, 1 parity bit, 1 stop bit)



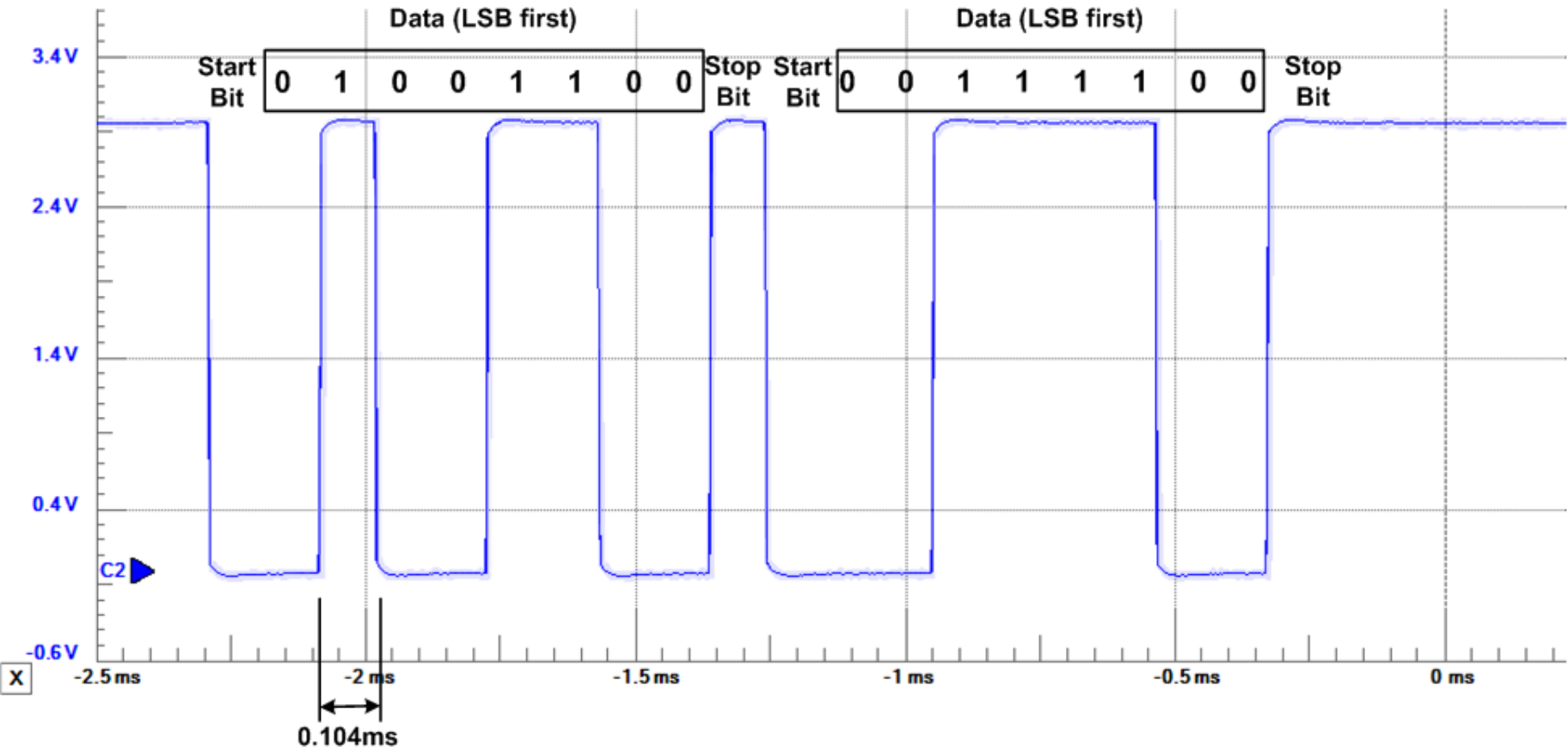
Tolerate 10% clock shift during transmission

- Sender and receiver uses the same transmission speed
- Data frame
 - One start bit
 - Data (LSB first or MSB first, and size of 7, 8, 9 bits)
 - Optional parity bit
 - One or two stop bit

Error Detection

- **Even Parity**: total number of “1” bits in data and parity is even
- **Odd Parity**: total number of “1” bits in data and parity is odd
- Example: Data = 10101011 (five “1” bits)
 - The parity bit should be **0** for odd parity and **1** for even parity
- This can detect **single-bit** data corruption

Transmitting 0x32 and 0x3C



1 start bit, 1 stop bit, 8 data bits, no parity, baud rate = 9600

Baud Rate

- Historically used in telecommunication to represent the number of pulses physically transferred per second
- In digital communication, baud rate is the number of bits physically transferred per second
- Example:
 - Baud rate is 9600
 - each frame: a start bit, 8 data bits, a stop bit, and no parity bit.
 - Transmission rate of actual data
 - ~~$9600/8 = 1200 \text{ bytes/second}$~~
 - $9600/(1 + 8 + 1) = 960 \text{ bytes/second}$
 - The start and stop bits are the protocol overhead

Baud Rate

$$\text{Baud Rate} = \frac{(1 + \text{OVER8}) \times f_{PCLK}}{\text{USARTDIV}}$$

- If *OVER8* is 0, then the signal is oversampled by 16, and 4 bits are used for the fractional part.
- If *OVER8* is 1, then the signal is oversampled by 8, and 3 bits are used.
- USARTDIV refers to the contents of the BRR register

Baud Rate Example 1

- Determine USARTDIV (BRR contents) necessary for baud=14400, given that OVER8=0, processor clock = 16 MHz.
- $$USARTDIV = \frac{(1+OVER8) \times f_{PCLK}}{Baud\ Rate}$$
- $$USARTDIV = \frac{(1+0) \times 16000000}{14400}$$
- $$USARTDIV \approx 1111 = 0x457$$

Baud Rate Example 2

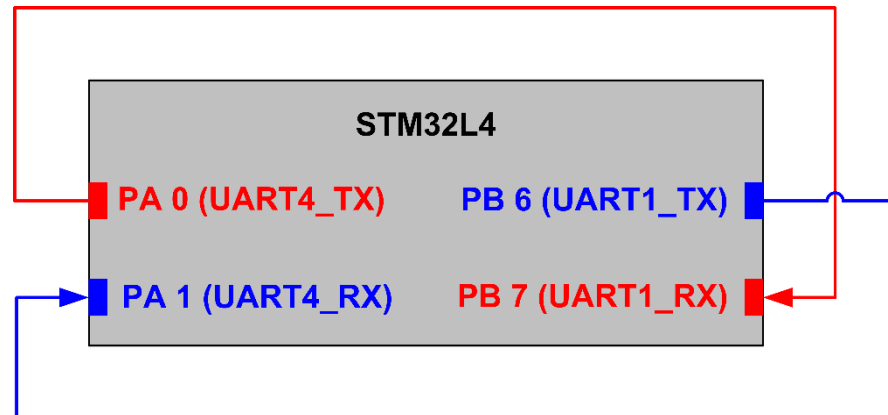
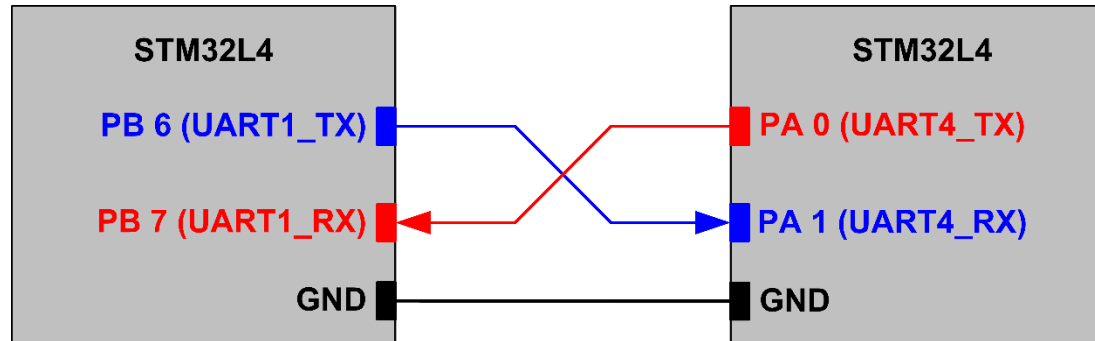
- Determine USARTDIV (BRR contents) necessary for baud=9600, given that OVER8=0, processor clock = 80 MHz.

- $$USARTDIV = \frac{(1+OVER8) \times f_{PCLK}}{Baud\ Rate}$$

- $$USARTDIV = \frac{(1+0) \times 80000000}{9600}$$

- $$USARTDIV \approx 8333 = 0x208D$$

UART Connection



Setting Up the Code

```
void USART_Init (USART_TypeDef * USARTx) {  
    // Disable USART  
    USARTx->CR1 &= ~USART_CR1_UE;  
  
    // Set data length to 8 bits  
    // 00 = 8 data bits, 01 = 9 data bits, 10 = 7 data bits  
    USARTx->CR1 &= ~USART_CR1_M;  
  
    // Select 1 stop bit  
    // 00 = 1 Stop bit      01 = 0.5 Stop bit  
    // 10 = 2 Stop bits    11 = 1.5 Stop bit  
    USARTx->CR2 &= ~USART_CR2_STOP;  
  
    // Set parity control as no parity  
    // 0 = no parity,  
    // 1 = parity enabled (then, program PS bit to select Even or Odd parity)  
    USARTx->CR1 &= ~USART_CR1_PCE;  
  
    // Oversampling by 16  
    // 0 = oversampling by 16, 1 = oversampling by 8  
    USARTx->CR1 &= ~USART_CR1_OVER8;  
  
    // Set Baud rate to 9600 using APB frequency (80 MHz)  
    // See Example 1 in Section 22.1.2  
    USARTx->BRR = 0x208D;  
  
    // Enable transmission and reception  
    USARTx->CR1 |= (USART_CR1_TE | USART_CR1_RE);  
  
    // Enable USART  
    USARTx->CR1 |= USART_CR1_UE;  
  
    // Verify that USART is ready for transmission  
    // TEACK: Transmit enable acknowledge flag. Hardware sets or resets it.  
    while ((USARTx->ISR & USART_ISR_TEACK) == 0);  
  
    // Verify that USART is ready for reception  
    // REACK: Receive enable acknowledge flag. Hardware sets or resets it.  
    while ((USARTx->ISR & USART_ISR_REACK) == 0);  
}
```

Setting Up the Code

```
int main(void){
    // Enable GPIO clock and configure the Tx pin and the Rx pin as:
    // Alternate function, High Speed, Push-pull, Pull-up

    //----- GPIO Initialization for USART 1 -----
    // PB.6 = AF7 (USART1_TX), PB.7 = AF7 (USART1_RX)
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN; // Enable GPIO port B clock

    // 00 = Input, 01 = Output, 10 = Alternate Function, 11 = Analog
    GPIOB->MODER  &= ~(0xF << (2*6)); // Clear mode bits for pin 6 and 7
    GPIOB->MODER  |=  0xA << (2*6); // Select Alternate Function mode

    // Alternative function 7 = USART 1
    // Appendix I shows all alternate functions
    GPIOB->AFR[0] |=  0x77 << (4*6); // Set pin 6 and 7 to AF 7

    // GPIO Speed: 00 = Low speed, 01 = Medium speed,
    //              10 = Fast speed, 11 = High speed
    GPIOB->OSPEEDR |=  0xF << (2*6);

    // GPIO Push-Pull: 00 = No pull-up/pull-down, 01 = Pull-up (01)
    //                  10 = Pull-down, 11 = Reserved
    GPIOB->PUPDR  &= ~(0xF << (2*6));
    GPIOB->PUPDR  |=  0x5 << (2*6); // Select pull-up

    // GPIO Output Type: 0 = push-pull, 1 = open drain
    GPIOB->OTYPER &= ~(0x3 << 6);

    //----- GPIO Initialization for USART 4 -----
    // PA.0 = AF8 (UART4_TX), PA.1 = AF8 (UART4_RX)
    // The code is very similar to the one given above.
    ...

    RCC->APB2ENR |= RCC_APB2ENR_USART1EN; // Enable UART 1 clock
    RCC->APB1ENR1 |= RCC_APB1ENR1_UART4EN; // Enable UART 4 clock

    // Select system clock (SYSCLK) USART clock source of UART 1 and 4
    // 00 = PCLK, 01 = System clock (SYSCLK),
    // 10 = HSI16, 11 = LSE
    RCC->CCIPR &= ~(RCC_CCIPR_USART1SEL | RCC_CCIPR_UART4SEL);
    RCC->CCIPR |= (RCC_CCIPR_USART1SEL_0 | RCC_CCIPR_UART4SEL_0);

    USART_Init(USART1);
    USART_Init(UART4);
    ...
}
```

Example 22-2. Enable and select the clock of UART ports

Processor's Pins

Joystick (MT-008A): Center = PA0, Left = PA1, Right = PA2, Up = PA3, Down = PA5

User LED: Red = PB2, Green = PE8

Analog Outputs: PA3 (OPAMP1_OUT), PA5 (DAC1_OUT2). Note: PA4 (DAC1_OUT1) is not extended out, thus we use OPAMP1_OUT.

Joystick (center)	PA0/TIM2_CH1/TIM5_CH1/TIM8_ETR/USART2_CTS/UART4_TX/SAI1_EXTCLK/TIM2_ETR	PC9/TIM8_BKIN/TIM3_CH4/TIM8_CH4/TSC_G4_I04/OTG_FS_NOE/LCD_SEG27/SDMMC1_D1/SAI2_EXTCLK/TIM8_BKIN2_COMP1	OTG_FS_PowerSwitchOn
Joystick (left)	PA1/TIM2_CH2/TIM5_CH2/USART2_RTS_DE/UART4_RX/LCD_SEG0/TIM15_CH1N	PC10/SPI3_SCK/USART3_TX/UART4_TX/TSC_G3_I02/LCD_COM4/LCD_SEG28/LCD_SEG40/SDMMC1_D2/SAI2_SCK_B	OTG_FS_OverCurrent
Joystick (right)	PA2/TIM2_CH3/TIM5_CH3/USART2_TX/LCD_SEG1/SAI2_EXTCLK/TIM15_CH1	PC11/SPI3_MISO/USART3_RX/UART4_RX/TSC_G3_I03/LCD_COM5/LCD_SEG29/LCD_SEG41/SDMMC1_D3/SAI2_MCLK_B	OTG_FS_VBUS
Joystick (up)	PA3/TIM2_CH4/TIM5_CH4/USART2_RX/LCD_SEG2/TIM15_CH2	PC12/SPI3_MOSI/USART3_CK/UART5_TX/TSC_G3_I04/LCD_COM6/LCD_SEG30/LCD_SEG42/SDMMC1_CK/SAI2_SD_B	OTG_FS_ID
Joystick (down)	PA4/SPI1_NSS/SPI3_NSS/USART2_CK/SAI1_FS_B/LPTIM2_OUT	PC13/PD0/SPI2_NSS/DFSDM_DATIN7/CAN1_RX/FMC_D2	OSC32_IN
LCD SEG23	PA5/TIM2_CH1/TIM2_ETR/TIM8_CH1N/SPI1_SCK/LPTIM2_ETR	PC14/	OSC32_OUT
LCD SEG0	PA6/TIM1_BKIN/TIM3_CH1/TIM8_BKIN/SPI1_MISO/USART3_CTS/QUADSPI_BK1_I03/LCD_SEG3/TIM1_BKIN_COMP2/TIM8_BKIN_COMP2/TIM16_CH1	PC15/	
LCD COM0	PA7/TIM1_CH1N/TIM3_CH2/TIM8_CH1N/SPI1_MOSI/QUADSPI_BK1_I02/LCD_SEG4/TIM17_CH1		
LCD COM1	PA8/MCO/TIM1_CH1/USART1_CK/OTG_FS_SOF/LCD_COM0/LPTIM2_OUT	PD1/SPI2_SCK/DFSDM_CKIN7/CAN1_TX/FMC_D3	eCompass/Gyro MEMS_SCK
LCD COM2	PA9/TIM1_CH2/USART1_TX/LCD_COM1/TIM15_BKIN	PD2/TIM3_ETR/USART3_RTS_DE/UART5_RX/TSC_SYNC/LCD_COM7/LCD_SEG31/LCD_SEG43/SDMMC1_CMD	Gyro GYRO_INT1
OTG_FS_DM	PA10/TIM1_CH3/USART1_RX/OTG_FS_ID/LCD_COM2/TIM17_BKIN	PD3/SPI2_MISO/DFSDM_DATIN0/USART2_CTS/FMC_CLK	Gyro MEMS_MISO
OTG_FS_DP	PA11/TIM1_CH4/TIM1_BKIN2/USART1_CTS/CAN1_RX/OTG_FS_DM/TIM1_BKIN2_COMP1	PD4/SPI2_MOSI/DFSDM_CKIN0/USART2_RTS_DE/FMC_NOE	eCompass/Gyro MEMS_MOSI
ST-Link SWDIO	PA12/TIM1_ETR/USART1_RTS_DE/CAN1_TX/OTG_FS_DP	PD5/USART2_TX/FMC_NWE	ST-Link USART_TX
ST-Link SWCLK	PA13/JTMS/SWDIO/IIR_OUT/OTG_FS_NOE	PD6/DFSDM_DATIN1/FMC_NWAIT/SAI1_SD_A	ST-Link USART_RX
LCD SEG10	PA14/JTCK/SWCLK	PD7/DFSDM_CKIN1/USART2_CK/FMC_NE1	Gyro GYRO_CS
	PA15/JTDI/TIM2_CH1/TIM2_ETR/SPI1_NSS/SPI3_NSS/UART4_RTS_DE/TSC_G3_I01/LCD_SEG17/SAI2_FS_B	PD8/USART3_TX/LCD_SEG28/FMC_D13	LCD SEG18
LCD SEG21	PB0/TIM1_CH2N/TIM3_CH3/TIM8_CH2N/USART3_CK/QUADSPI_BK1_I01/LCD_SEG5/COMP1_OUT	PD9/USART3_RX/LCD_SEG29/FMC_D14/SAI2_MCLK_A	LCD SEG5
LCD SEG2	PB1/TIM1_CH3N/TIM3_CH4/TIM8_CH3N/DFSDM_DATIN0/USART3_RTS_DE/QUADSPI_BK1_I00/LCD_SEG6/LPTIM2_IN1	PD10/USART3_CK/TSC_G6_I01/LCD_SEG30/FMC_D15/SAI2_SCK_A	LCD SEG17
LED red	PB2/RTC_OUT/LPTIM1_OUT/I2C3_SMBA/DFSDM_CKIN0	PD11/USART3_CTS/TSC_G6_I02/LCD_SEG31/FMC_A16/SAI2_SD_A/LPTIM2_ETR	LCD SEG6
ST-Link SWO	PB3/JTDO/TRACESWO/TIM2_CH2/SPI3_SCK/USART1_RTS_DE/LCD_SEG7/SAI1_SCK_B	PD12/TIM4_CH1/USART3_RTS_DE/TSC_G6_I03/LCD_SEG32/FMC_A17/SAI2_FS_A/LPTIM2_IN1	LCD SEG16
LCD SEG11	PB4/NTRST/TIM3_CH1/SPI1_MISO/SPI3_MOSI/USART1_CTS/UART5_RTS_DE/TSC_G2_I01/LCD_SEG8/SAI1_MCLK_B/TIM17_BKIN	PD13/TIM4_CH2/TSC_G6_I04/LCD_SEG33/FMC_A18/LPTIM2_OUT	LCD SEG7
Codec I2C1_SCL	PB5/LPTIM1_IN1/TIM3_CH2/I2C1_SMBB/SPI1_MOSI/SPI3_MOSI/USART1_CTS/UART5_TX/TSC_G2_I02/LCD_SEG9/COMP2_OUT/SAI1_SD_B/TIM16_BKIN	PD14/TIM4_CH3/LCD_SEG34/FMC_D0	LCD SEG15
Codec I2C1_SDA	PB6/LPTIM1_ETR/TIM4_CH1/TIM8_BKIN/I2C1_SCL/DFSDM_DATIN5/USART1_TX/TSC_G2_I03/TIM8_BKIN2_COMP2/SAI1_FS_B/TIM16_CH1N	PD15/TIM4_CH4/LCD_SEG35/FMC_D1	LCD SEG8
Gyro GYRO_INT1	PB7/LPTIM1_IN2/TIM4_CH2/TIM8_BKIN/I2C1_SDA/DFSDM_CKIN1/USART1_RX/UART4_CTS/TSC_G2_I04/LCD_SEG21/FMC_NL/TIM8_BKIN_COMP1/TIM17_CH1N		
LCD COM3	PB8/TIM4_CH3/I2C1_SCL/DFSDM_DATIN6/CAN1_RX/LCD_SEG1/SDMMC1_D4/SAI1_CLK_A/TIM16_CH1	PE0/TIM4_ETR/LCD_SEG36/FMC_NBL0/TIM16_CH1	eCompass MAG_INT
	PB9/IIR_OUT/TIM4_CH4/I2C1_SDA/SPI2_NSS/DFSDM_CKIN6/CAN1_TX/LCD_COM3/SDMMC1_D5/SAI1_FS_A/TIM17_CH1	PE1/LCD_SEG37/FMC_NBL1/TIM17_CH1	eCompass XL_INT
LCD SEG20	PB10/TIM2_CH3/I2C2_SCL/SPI2_SCK/DFSDM_DATIN7/USART3_TX/LPUART1_RX/QUADSPI_CLK/LCD_SEG10/COMP1_OUT/SAI1_SCK_A	PE2/TRACECK/TIM3_ETR/TSC_G7_I01/LCD_SEG38/FMC_A23/SAI1_MCLK_A	Codec SAH1_MCK
LCD SEG3	PB11/TIM2_CH4/I2C2_SDA/DFSDM_CKIN7/USART3_RX/LPUART1_TX/QUADSPI_NCS/LCD_SEG11/COMP2_OUT	PE3/TRACED0/TIM3_CH1/TSC_G7_I02/LCD_SEG39/FMC_A19/SAI1_SD_B	Codec Audio_RST
LCD SEG19	PB12/TIM1_BKIN/TIM1_BKIN_COMP2/I2C2_SMBB/SPI2_NSS/DFSDM_DATIN1/USART3_CK/LPUART1_RTS_DE/TSC_G1_I01/LCD_SEG12/SWPIM1_I0/SAI2_FS_A/TIM15_BKIN	PE4/TRACED1/TIM3_CH2/DFSDM_DATIN3/TSC_G7_I03/FMC_A20/SAI1_FS_A	Codec SAH1_FS
LCD SEG4	PB13/TIM1_CH1N/I2C2_SCL/SPI2_SCK/DFSDM_CKIN1/USART3_CTS/LPUART1_CTS/TSC_G1_I02/LCD_SEG13/SWPIM1_TX/SAI2_SCK_A/TIM15_CH1N	PE5/TRACED2/TIM3_CH3/DFSDM_CKIN3/TSC_G7_I04/FMC_A21/SAI1_SCK_A	Codec SAH1_SCK
	PB14/TIM1_CH2N/TIM8_CH2N/I2C2_SDA/SPI2_MISO/DFSDM_DATIN2/USART3_RTS_DE/TSC_G1_I03/LCD_SEG14/SWPIM1_RX/SAI2_MCLK_A/TIM15_CH1	PE6/TRACED3/TIM3_CH4/FMC_A22/SAI1_SD_A	Codec SAH1_SD
	PB15/RTC_REFIN/TIM1_CH3N/TIM8_CH3N/SPI2_MOSI/DFSDM_CKIN2/TSC_G1_I04/LCD_SEG15/SWPIM1_SUSPEND/SAI2_SD_A/TIM15_CH2	PE7/TIM1_ETR/DFSDM_DATIN2/FMC_D4/SAI1_SD_B	Audio_DIN
eCompass MAG_CS	PC0/LPTIM1_IN1/I2C3_SCL/DFSDM_DATIN4/LPUART1_RX/LCD_SEG18/LPTIM2_IN1	PE8/TIM1_CH1N/DFSDM_CKIN2/FMC_D5/SAI1_SCK_B	Audio_CLK
eCompass MAG_INT	PC1/LPTIM1_OUT/I2C3_SDA/DFSDM_CKIN4/LPUART1_TX/LCD_SEG19	PE9/TIM1_CH1/DFSDM_CKOUT/FMC_D6/SAI1_FS_B	LED green
eCompass MAG_DRDY	PC2/LPTIM1_IN2/SPI2_MISO/DFSDM_CKOUT/LCD_SEG20	PE10/TIM1_CH2N/DFSDM_DATIN4/TSC_G5_I01/QUADSPI_CLK/FMC_D7/SAI1_MCLK_B	Flash QSPI_CLK
LCD VLCD	PC3/LPTIM1_ETR/SPI2_MOSI/LCD_VLCD/SAI1_SD_A/LPTIM2_ETR	PE11/TIM1_CH2/DFSDM_CKIN4/TSC_G5_I02/QUADSPI_NCS/FMC_D8	Flash QSPI_D0
LCD SEG22	PC4/USART3_TX/LCD_SEG22	PE12/TIM1_CH3N/SPI1_NSS/DFSDM_DATIN5/TSC_G5_I03/QUADSPI_BK1_I00/FMC_D9	Flash QSPI_D1
LCD SEG1	PC5/USART3_RX/LCD_SEG23	PE13/TIM1_CH3/SPI1_SCK/DFSDM_CKIN5/TSC_G5_I04/QUADSPI_BK1_I01/FMC_D10	Flash QSPI_D2
LCD SEG14	PC6/TIM3_CH1/TIM8_CH1/DFSDM_CKIN3/TSC_G4_I01/LCD_SEG24/SDMMC1_D6/SAI2_MCLK_A	PE14/TIM1_CH4/TIM1_BKIN2_COMP2/SPI1_MISO/QUADSPI_BK1_I02/FMC_D11	Flash QSPI_D3
LCD SEG9	PC7/TIM3_CH2/TIM8_CH2/DFSDM_DATIN3/TSC_G4_I02/LCD_SEG25/SDMMC1_D7/SAI2_MCLK_B	PE15/TIM1_BKIN/TIM1_BKIN_COMP1/SPI1_MOSI/QUADSPI_BK1_I03/FMC_D12	
LCD SEG13	PC8/TIM3_CH3/TIM8_CH3/TSC_G4_I03/LCD_SEG26/SDMMC1_D0		OSC_IN
			OSC_OUT
			PH0/
			PH1/

STM32L476VG Discovery Kit Pin Connection

Table 9. Alternate function mapping

Port		AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7	AF8	AF9	AF10	AF11	AF12	AF13	AF14	AF15
		SYS	TIM1/2	TIM3/4/5	TIM8/9/10/11	I2C1/2/3	SPI1/SPI2/I2S2/I2S2ext	SPI3/I2S3ext/I2S3	USART1/2/3/I2S3ext	UART4/5/USART6	CAN1/2/TIM12/13/14	OTG_FS/OTG_HS	ETH	FSMC/SDIO/OTG_FS	DCMI		
Port A	PA0	-	TIM2_CH1_ETR	TIM5_CH1	TIM8_ETR	-	-	-	USART2_CTS	UART4_TX	-	-	ETH_MII_CRS	-	-	-	EVENTOUT
	PA1	-	TIM2_CH2	TIM5_CH2	-	-	-	-	USART2_RTS	UART4_RX	-	-	ETH_MII_RX_CLK ETH_RMII_REF_CLK	-	-	-	EVENTOUT
	PA2	-	TIM2_CH3	TIM5_CH3	TIM9_CH1	-	-	-	USART2_TX	-	-	-	ETH_MDIO	-	-	-	EVENTOUT
	PA3	-	TIM2_CH4	TIM5_CH4	TIM9_CH2	-	-	-	USART2_RX	-	-	OTG_HS_ULPI_D0	ETH_MII_COL	-	-	-	EVENTOUT
	PA4	-	-	-	-	-	SPI1_NSS	SPI3_NSS/I2S3_WS	USART2_CK	-	-	-	-	OTG_HS_SOF	DCMI_HSYNC	-	EVENTOUT
	PA5	-	TIM2_CH1_ETR	-	TIM8_CH1N	-	SPI1_SCK	-	-	-	-	OTG_HS_ULPI_CK	-	-	-	-	EVENTOUT
	PA6	-	TIM1_BKIN	TIM3_CH1	TIM8_BKIN	-	SPI1_MISO	-	-	-	TIM13_CH1	-	-	-	DCMI_PIXCLK	-	EVENTOUT
	PA7	-	TIM1_CH1N	TIM3_CH2	TIM8_CH1N	-	SPI1_MOSI	-	-	-	TIM14_CH1	-	ETH_MII_RX_DV ETH_RMII_CRS_DV	-	-	-	EVENTOUT
	PA8	MCO1	TIM1_CH1	-	-	I2C3_SCL	-	-	USART1_CK	-	-	OTG_FS_SOF	-	-	-	-	EVENTOUT
	PA9	-	TIM1_CH2	-	-	I2C3_SMBA	-	-	USART1_TX	-	-	-	-	-	DCMI_D0	-	EVENTOUT
	PA10	-	TIM1_CH3	-	-	-	-	-	USART1_RX	-	-	OTG_FS_ID	-	-	DCMI_D1	-	EVENTOUT
	PA11	-	TIM1_CH4	-	-	-	-	-	USART1_CTS	-	CAN1_RX	OTG_FS_DM	-	-	-	-	EVENTOUT
	PA12	-	TIM1_ETR	-	-	-	-	-	USART1_RTS	-	CAN1_TX	OTG_FS_DP	-	-	-	-	EVENTOUT
	PA13	JTMS-SWDIO	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENTOUT
	PA14	JTCK-SWCLK	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENTOUT
	PA15	JTDI	TIM2_CH1 TIM2_ETR	-	-	-	SPI1_NSS	SPI3_NSS/I2S3_WS	-	-	-	-	-	-	-	-	EVENTOUT

Pinouts and pin description

STM32F405xx, STM32F407xx

Setting Up the Code

method is inefficient, and the while loop prevents the processor from running other tasks.

```
void USART_Read (USART_TypeDef *USARTx, uint8_t *buffer, uint32_t nBytes) {  
    int i;  
  
    for (i = 0; i < nBytes; i++) {  
        while (!(USARTx->ISR & USART_ISR_RXNE)); // Wait until hardware sets RXNE  
        buffer[i] = USARTx->RDR; // Reading RDR clears RXNE  
    }  
}
```

Example 22-3. Receive data from a UART port by using busy polling

When UART sends a byte, it sets the RXNE flag in the USART_ISR register.

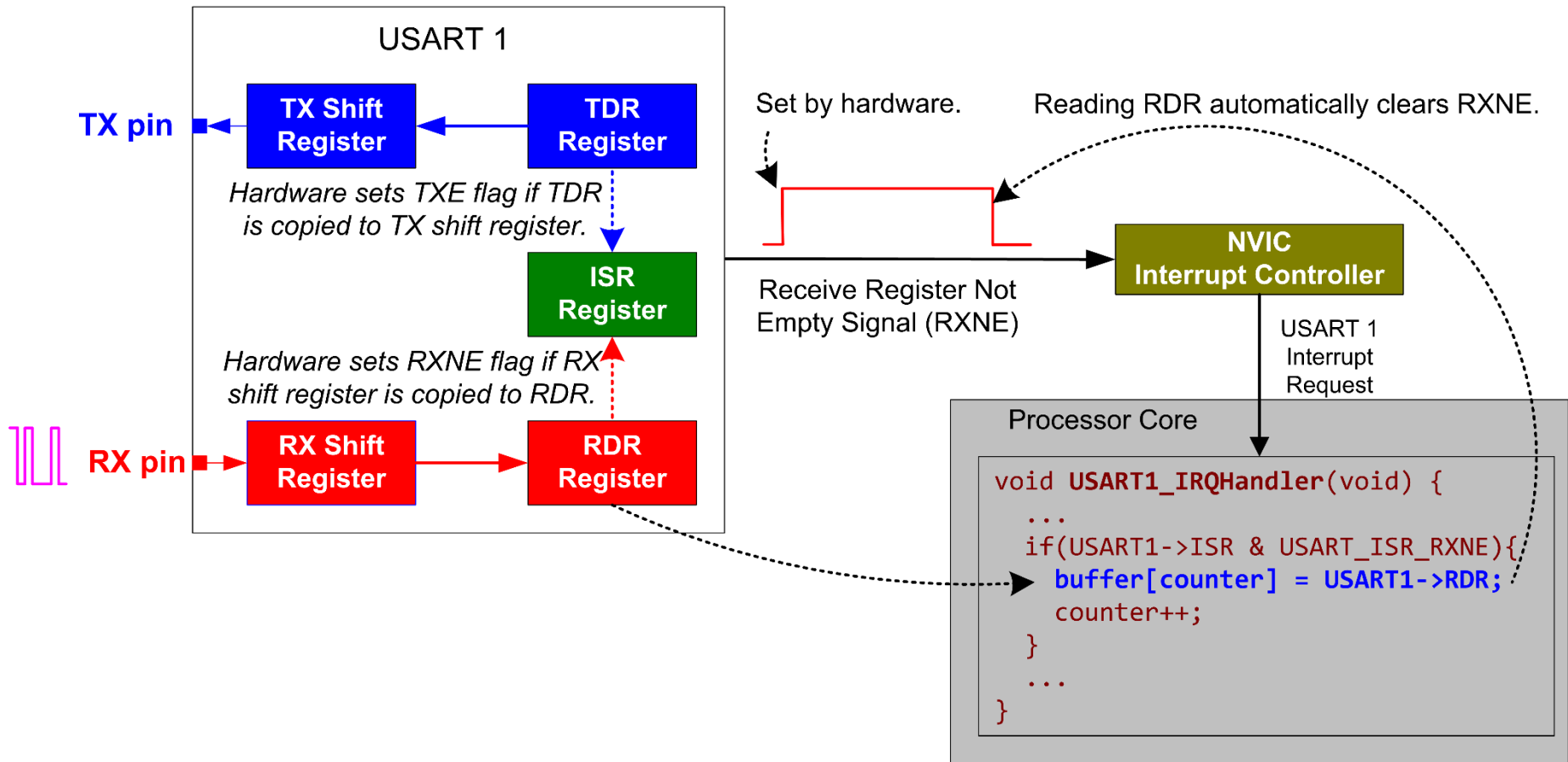
Setting Up the Code

loop prevents the processor from performing other tasks.

```
void USART_Write (USART_TypeDef *USARTx, uint8_t *buffer, uint32_t nBytes) {  
  
    int i;  
  
    for (i = 0; i < nBytes; i++) {  
        while (!(USARTx->ISR & USART_ISR_TXE)); // Wait until hardware sets TXE  
        USARTx->TDR = buffer[i] & 0xFF;           // Writing to TDR clears TXE flag  
    }  
  
    // Wait until TC bit is set. TC is set by hardware and cleared by software.  
    while (!(USARTx->ISR & USART_ISR_TC));         // TC: Transmission complete flag  
  
    // Writing 1 to the TCCF bit in ICR clears the TC bit in ISR  
    USARTx->ICR |= USART_ICR_TCCF; // TCCF: Transmission complete clear flag  
}
```

Example 22-4. Send data out via a UART port via busy polling

UART Interrupt: Receiving Data



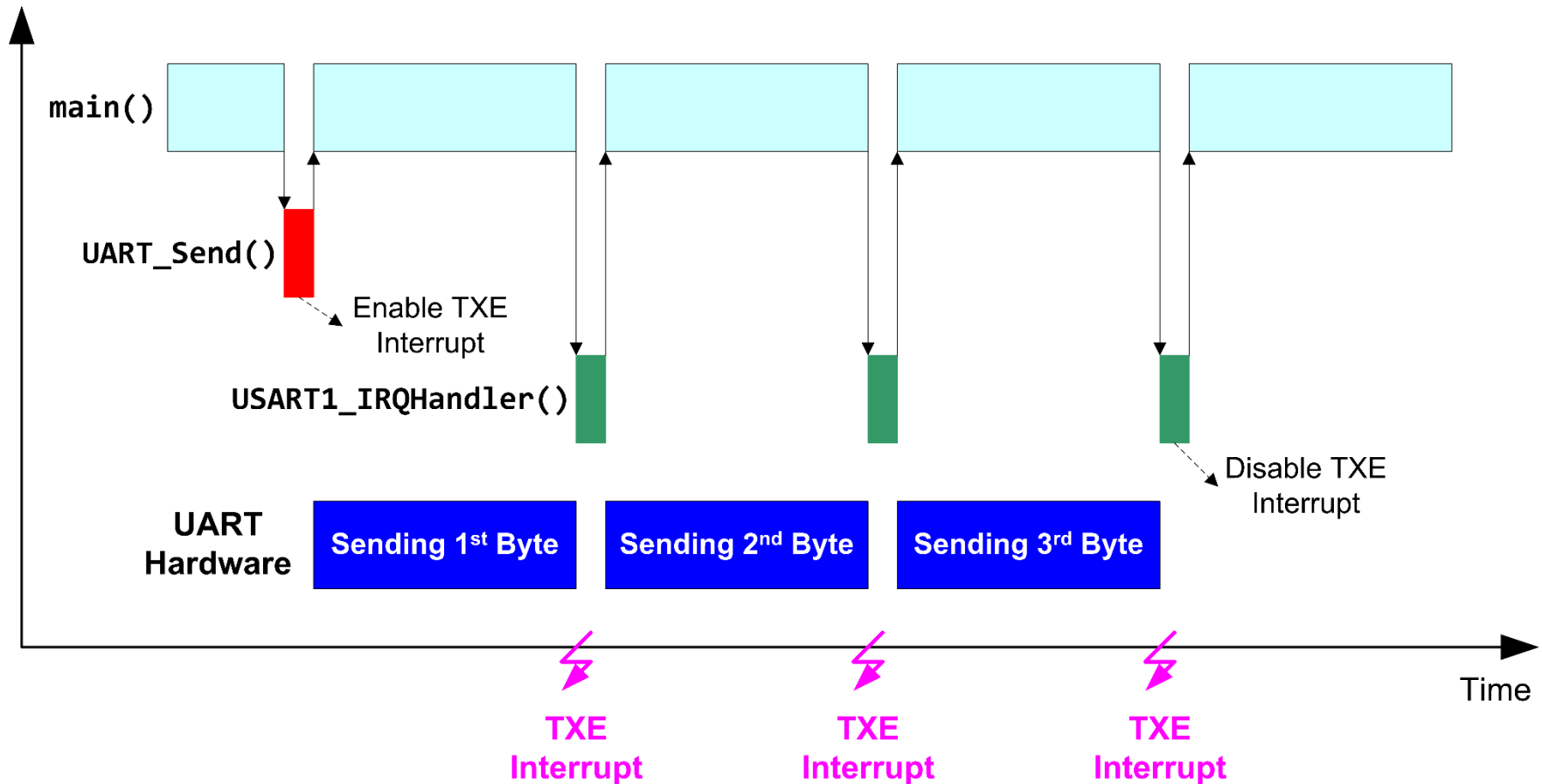
UART Interrupt: Receiving Data

```
void USART_IRQHandler(USART_TypeDef * USARTx, uint8_t * buffer,  uint8_t * pRx_counter){
    if(USARTx->SR & USART_SR_RXNE) { // Received data
        buffer[*pRx_counter] = USARTx->DR;
        // Reading USART_DR automatically clears the RXNE flag
        (*pRx_counter)++;
        if((*pRx_counter) >= BufferSize )
            (*pRx_counter) = 0;
    }
}

void USART1_IRQHandler(void) {
    USART_IRQHandler(USART1, USART1_Buffer_Rx, &Rx1_Counter);
}

void USART2_IRQHandler(void) {
    USART_IRQHandler(USART2, USART2_Buffer_Rx, &Rx2_Counter);
}
```


UART Interrupt: Sending Data



ASCII

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Source: www.LookUpTables.com

Voltage Levels

Standard	Voltage signal	Max distance	Max speed	Number of devices supported per port
RS-232	Single end (logic 1: +3 to +15V, logic 0: -3 to -15 V)	100 feet	115Kbit/s	1 master, 1 receiver
RS-422	Differential (-6V to +6V)	4000 feet	10Mbit/s	1 master, 10 receivers
RS-485	Differential (-7V to +12V)	4000 feet	10Mbit/s	32 masters, 32 receivers